

Architecture Document

AI-Powered Industrial Safety Intelligence Platform

Sahyadri Petrochem & Specialty Chemicals Ltd. (SPSCL) — Simulated Facility

ET AI Hackathon 2026 — Problem Statement #1-AI-Powered Industrial Safety Intelligence Platform

Submitted by Aditi Anand Kumar

github.com/pixel-pudding/Industrial-Safety-Intelligence

Table of Contents

1. Purpose & Scope
2. High-Level System Architecture
3. Agent Orchestration Model
4. Data Flow — Sources to Decision
5. Data Schema
6. API Surface (REST + WebSocket)
7. Scenario 10 — Full Cascade Walkthrough
8. Design Rationale & Trade-offs
9. Scalability Considerations
10. Known Limitations

1. Purpose & Scope

This document describes the technical architecture of the SPSCl AI-Powered Industrial Safety Intelligence Platform — how its six AI agents are organized, how data moves from simulated plant sensors through reasoning to action, how the schema is structured, and what the API surface looks like. It complements the project's Detailed Project Document (problem, business impact, judging-criteria alignment) with the implementation-level detail an engineer or technical judge would want.

Scope: this document covers the backend (FastAPI, agents, simulator), the data layer (SQL + ChromaDB), and the API contract the frontend consumes. It does not re-cover frontend component structure, which is addressed separately in the repository README.

2. High-Level System Architecture

The system is organized in five layers: a simulation layer that generates realistic plant data, a FastAPI backend that exposes it, an AI agent layer that reasons over it, a data layer that persists structured records and the RAG corpus, and a React frontend that visualizes the result. Layers communicate through a documented contract (REST + WebSocket), not shared memory or direct imports — each layer is independently testable.

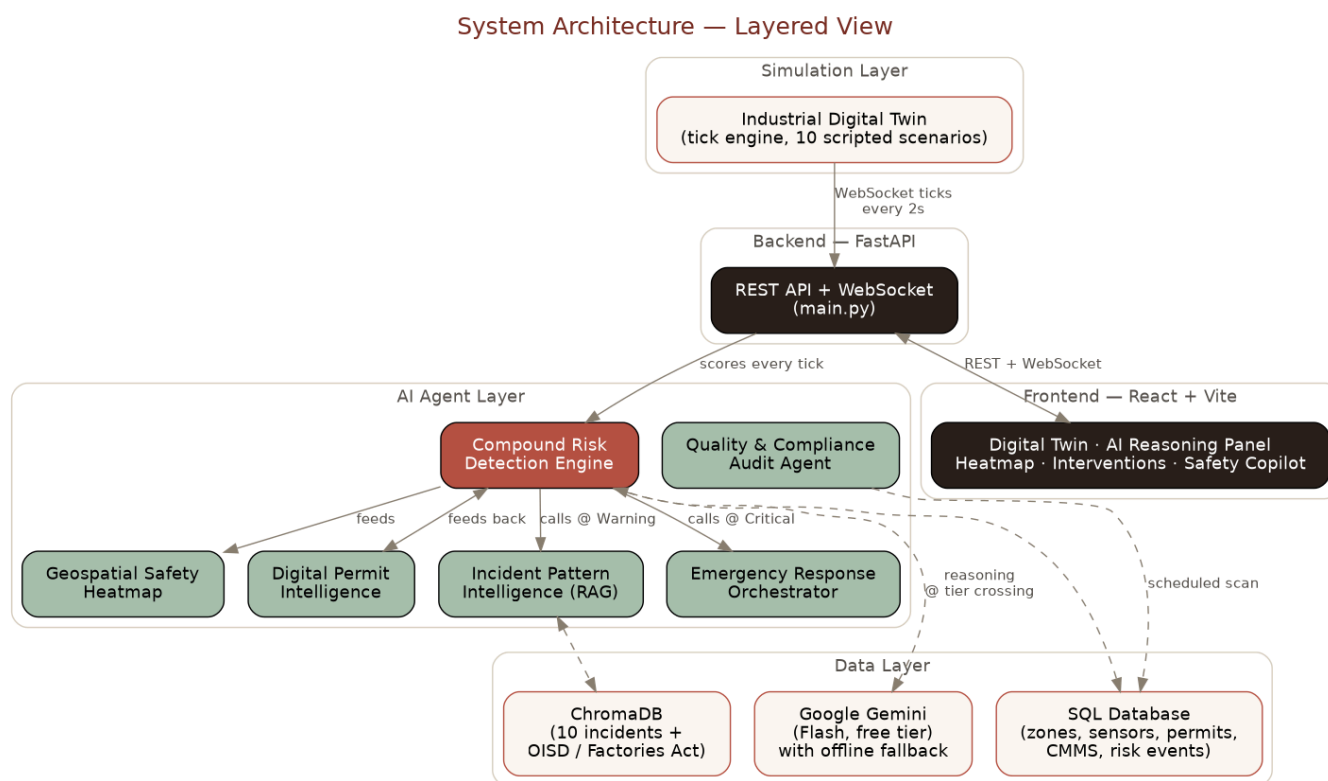


Figure 1 — Layered system architecture, from the simulated Digital Twin through to the dashboard.

The Digital Twin publishes a tick every 2 seconds over WebSocket. Every tick, the Compound Risk Detection Engine re-scores every zone using cheap, deterministic rule-based logic; only when a zone's tier crosses into Warning or Critical does the system call out to Gemini for the natural-language reasoning step — this keeps the system within Gemini's free-tier rate limit while still producing full explanations exactly when they matter.

3. Agent Orchestration Model

Agents are independent and specialized rather than one monolithic reasoning core, organized hub-and-spoke around the Compound Risk Detection Engine. This was a deliberate choice over a fully-meshed “every agent can call every agent” design: it is easier to test each agent in isolation, easier to reason about failure modes, and easier to explain to a judge or a new engineer joining the project.

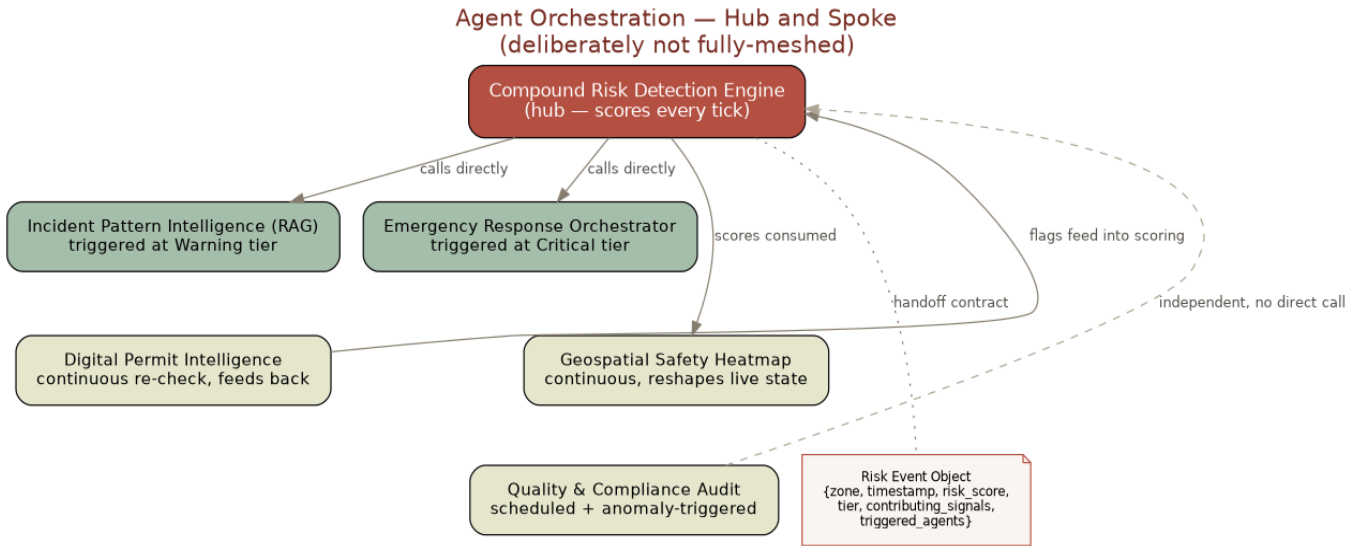


Figure 2 — Hub-and-spoke agent orchestration and the shared risk-event handoff contract.

Agent	Triggered By	Calls / Feeds
Compound Risk Detection Engine	Every SCADA/IoT tick	Calls Incident Pattern Intelligence (Warning) and Emergency Response (Critical)
Digital Permit Intelligence	Permit issuance + continuous re-check	Feeds flags back into the Detection Engine's score
Incident Pattern Intelligence (RAG)	Called at Warning tier	Returns matched incidents + regulatory citations into the evidence bundle
Emergency Response Orchestrator	Called at Critical tier	Reads the committed risk event; writes evacuation/alert/report records
Geospatial Safety Heatmap	Continuous	Reshapes Detection Engine + Permit Intelligence output; no independent logic
Quality & Compliance Audit	Scheduled / manual trigger	Independent — reads CMMS + permit tables directly, no agent-to-agent call

All agents that participate in the real-time path share one handoff object, the risk event: zone, timestamp, risk_score, tier, contributing_signals, and triggered_agents. This is the only data structure agents pass to each other — it keeps the contract small enough to document completely, which matters both for debugging and for explaining the system to evaluators.

4. Data Flow — Sources to Decision

Six independent data sources feed the rule-based scoring layer: SCADA/IoT sensor tags, the permit-to-work system, CMMS maintenance records, CCTV events, worker location, and shift logs. No single source can push a zone past Warning tier on its own except a hard-override case (a confirmed CCTV fire/smoke event, or a permit block-recommend) — everything else must combine with at least one other independent signal category, which is the literal mechanism behind “compound risk detection.”

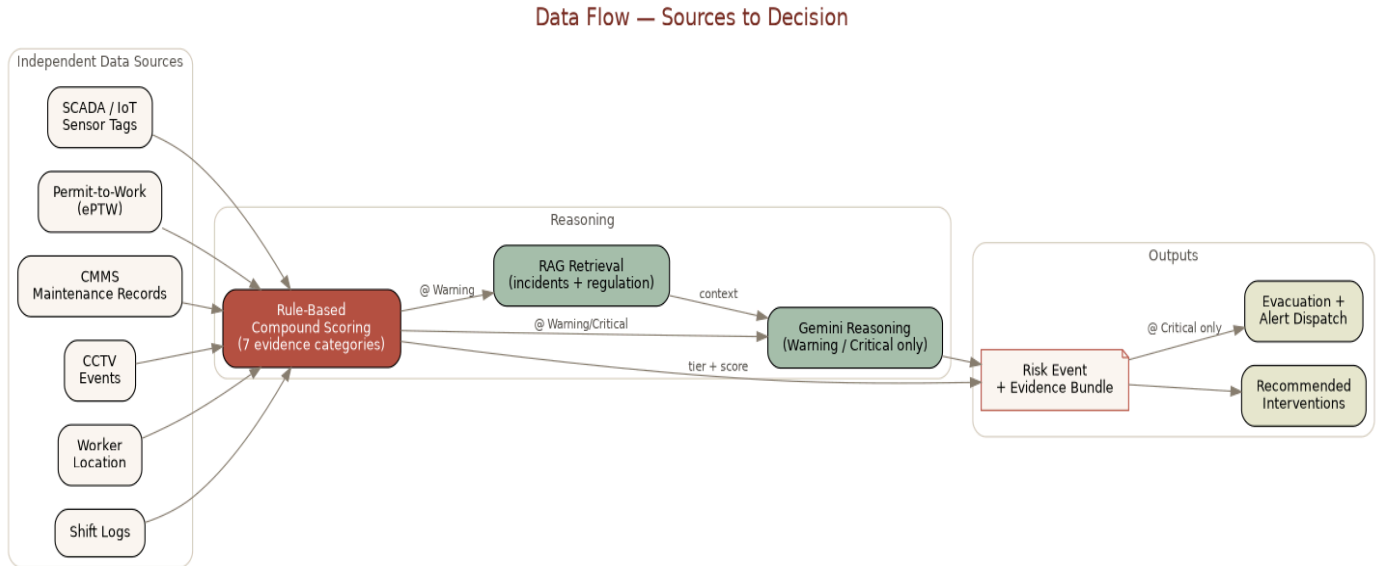


Figure 3 — Data flow from independent sources through scoring and retrieval to actionable output.

The Gemini reasoning step and the RAG retrieval step are both gated to fire only once a zone has already crossed into Warning or Critical by rule-based scoring alone. This ordering is intentional: the expensive/rate-limited steps (LLM call, vector search) only run when there is already a rule-grounded reason to explain, never on every tick for every zone.

5. Data Schema

The schema is relational (SQLAlchemy ORM) for everything structured — zones, assets, sensor readings, permits, maintenance, worker locations, CCTV events, and risk events — with one bridge column (narrative_doc_ref on historical_incidents) pointing into ChromaDB, where the free-text incident narratives and regulatory excerpts actually live for semantic retrieval.

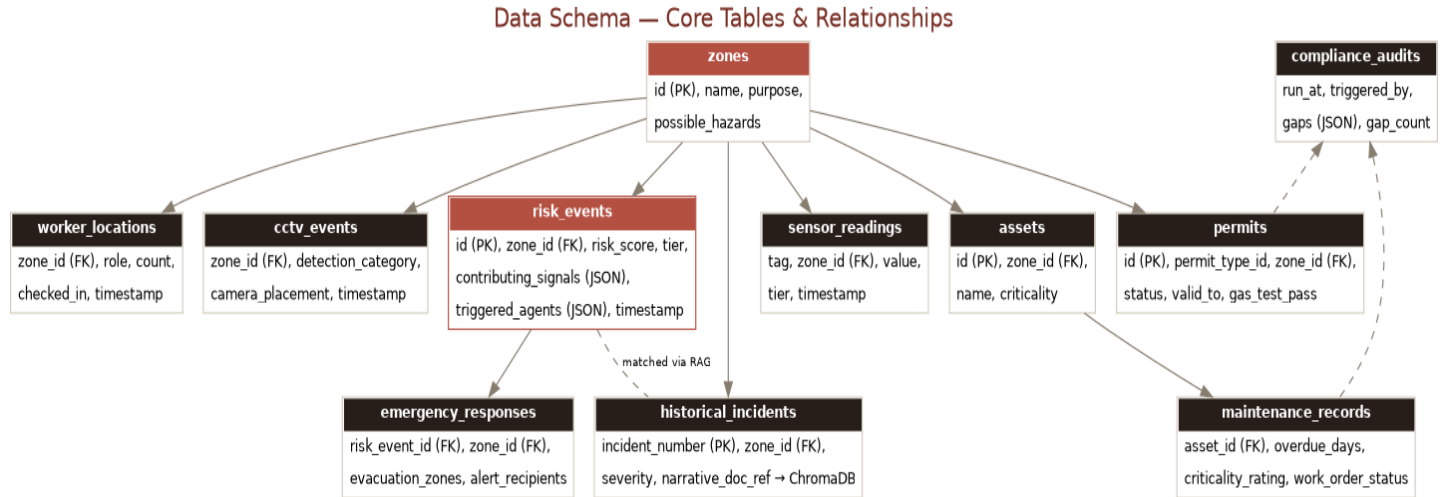


Figure 4 — Core tables and their relationships. *risk_events* is the central record every downstream agent reads from or writes to.

Two design choices worth calling out: `contributing_signals` and `triggered_agents` on `risk_events` are stored as JSON rather than fully normalized, since their shape varies per event (a sensor-only event and a 5-source compound event don't have the same signal count) — normalizing them would require a variable-width join table for no real query benefit at this scale. Second, `historical_incidents` keeps structured fields (zone, severity, hazard category) in SQL specifically so the RAG agent can pre-filter structurally before falling back to semantic search — this measurably improves retrieval precision over pure vector search alone.

6. API Surface (REST + WebSocket)

Endpoint	Purpose
GET /api/zones	Reference data — zone list, purpose, hazards
GET /api/scenarios	List the 10 demo scenarios
POST /api/scenario/{id}/run	Trigger a scenario, optional playback speed
POST /api/reset	Reset simulator + all agent state to baseline
GET /api/risk-events	Compound Risk Detection Engine history
GET /api/permits	Permit records, filterable by zone/status
GET /api/permits/flags	Digital Permit Intelligence — live flags
GET /api/emergency-responses	Emergency Response Orchestrator history
GET /api/heatmap	Geospatial Safety Heatmap — live zone view
POST /api/compliance-audit/run	Trigger Quality & Compliance Audit
GET /api/compliance-audit	Compliance audit history
GET /api/incidents	Historical incident corpus (structured fields)
POST /api/copilot/chat	AI Safety Copilot — natural-language queries
WS /ws	Live tick stream, every 2s — see payload shape below

WebSocket Tick Payload

Every 2 seconds, /ws broadcasts the full simulator state. This is the only place the AI Reasoning Panel's evidence (confidence, matched incident, regulatory citation, reasoning, interventions) is available — it is not persisted to the risk_events table, so a frontend that wants to show reasoning after the fact must capture it from the tick stream as it arrives.

```
{
  type: "tick", sim_time, scenario_running, active_scenario_id,
  readings: { [tag]: { value, zone, tier } },
  notes: string[],
  risk_events: [
    { ...,
      evidence: {
        confidence, matched_incident_summary, matched_incident_confidence,
        regulatory_citation, reasoning, recommended_interventions, llm_backed
      } | null
    }
  ],
  permit_flags: [...], emergency_responses: [...], heatmap: { ... }
}
```

7. Scenario 10 — Full Cascade Walkthrough

Scenario 10 is the flagship demo scenario and the clearest illustration of the full agent chain operating together, from raw signal to a dispatched emergency response. It mirrors the fatal-incident pattern cited in the problem statement: no single one of its five contributing signals would trigger an alert alone.

Scenario 10 Execution Flow — Full Compound Cascade

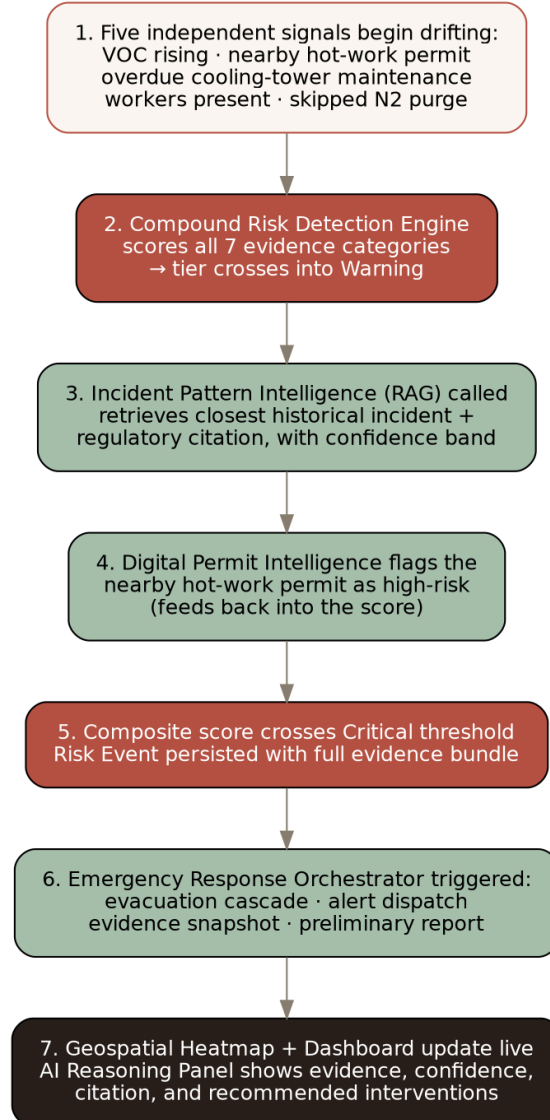


Figure 5 — Execution flow for Scenario 10, from signal onset to dashboard update.

Every step in this flow is produced by the actual running system when the scenario is triggered — none of it is scripted dashboard animation. This is verifiable by re-running the scenario after a reset and confirming byte-identical output, which was tested explicitly during development.

8. Design Rationale & Trade-offs

- Rule-based scoring first, LLM second: keeps the system inside Gemini's free-tier rate limit and makes the cheap, deterministic part of the reasoning auditable independent of any LLM call succeeding.
- Hub-and-spoke over fully-meshed agents: sacrifices some flexibility (an agent can't call any other agent directly) for a system that's dramatically easier to test, debug, and diagram for evaluators.
- Manually-triggered scenarios, not randomly auto-fired: chosen specifically because the deliverable is a recorded demo — manual triggering allows clean narration and re-recording of any scenario that doesn't play cleanly.
- Structured pre-filter before semantic RAG search: historical incidents carry structured zone/hazard/severity fields specifically so retrieval can narrow candidates before falling back to embeddings, which reduces the noise pure vector search shows with only 10 seeded incidents.
- JSON columns for variable-shape agent output (contributing_signals, triggered_agents) instead of forcing a rigid normalized schema that doesn't fit every event type.

9. Scalability Considerations

Adding a new zone, sensor tag, or permit type requires only a data-layer addition (a new row/seed entry) — no agent logic needs to change, since agents query by zone/tag pattern rather than hardcoding zone identifiers. Adding a new agent means adding one more spoke to the hub, not restructuring existing agents. The one component that would need real engineering work to scale past the free tier is the Gemini reasoning call — moving from personal API keys to a paid tier or a self-hosted model would remove the current rate-limit gating logic entirely, which is designed to be a no-op once that constraint is lifted.

10. Known Limitations

- RAG retrieval precision among closely related incidents is noisier than production-scale retrieval would be, with only 10 seeded historical incidents and a generic embedding model.
- Compliance Audit citations are lower-confidence for maintenance-interval gaps than for hot-work/confined-space gaps, since the regulatory corpus was scoped toward the latter.
- CCTV events are scripted JSON, not real computer-vision inference — by design, since the evaluation focus is reasoning over events, not perception accuracy.
- Evidence (reasoning, confidence, citations) is only available transiently in the WebSocket stream, not retrievable afterward from the REST risk-events endpoint — a frontend must capture it live.